

Falsify Before You Reward:

Interventional Verification for Reliable Reinforcement Learning

Anonymous Authors

Abstract

Reward-based learning depends on a verifier that distinguishes genuine task completion from behavior that merely satisfies an observed reward signal. Standard outcome verifiers, process verifiers, and even tool-augmented agentic verifiers primarily assess a submitted trajectory as observed. This creates an identifiability problem: two policies can produce the same successful observation while relying on fundamentally different mechanisms, including memorization, ignored dependencies, brittle shortcuts, or explicit reward exploits.

We introduce **Interventional Agentic Verification (IAV)**, a general verification framework that actively changes candidate-relevant variables, re-executes the system, and uses the resulting counterfactual evidence before assigning reward. IAV selects *discriminative interventions*: perturbations expected to separate competing hypotheses about whether success is faithful or shortcut-based. We formalize observational non-identifiability, show when interventions make policy classes distinguishable, and construct IAV-BENCH, an executable multi-domain benchmark spanning arithmetic reasoning, symbolic logic, executable code, shortcut policies, and reward-hacking environments.

Across eight-seed controlled evaluations, outcome-only verification exhibits false-positive rates of 83.5–100% on intentionally confounded tasks. Interventional verification reduces these errors sharply; for example, a single joint intervention separates faithful from shortcut policies with 100% accuracy in the controlled policy and code suites, while adaptive witness selection reaches 93.2% accuracy at budget two and 100% at budget three in symbolic logic. The benefit transfers to decision making: at Best-of-8, reliable candidate selection rises from 35.1% under outcome scoring to 96.8% with joint IAV, and unseen-perturbation success rises from 75.2% to 98.7%. In actual stochastic policy-gradient training over controlled solver strategies, IAV reward drives the faithful-policy probability from 35% to 99.6% after 800 REINFORCE updates, versus 36.6% under outcome reward. A key negative result is that arbitrary perturbation is insufficient: non-discriminative interventions can leave false-positive rates above 50%, motivating information-seeking witness selection.

These experiments isolate verifier properties rather than benchmark frontier language models. The results support a broader principle: *reliable reward should be based not only on whether a candidate succeeds, but on whether its success survives targeted attempts to falsify the mechanism that produced it.*

1 Introduction

Reinforcement learning with learned or programmatic feedback turns verification into an optimization target. Once a verifier’s score becomes reward, any systematic blind spot can become an incentive. This is the core difficulty behind specification gaming and reward hacking: an agent can satisfy the measurable criterion without accomplishing the intended task (Everitt et al., 2017; DeepMind, 2020). The same problem appears in modern reasoning systems. An answer may be numerically correct while its intermediate derivation is invalid; a program may pass visible tests while depending on a brittle shortcut; an agent may trigger the terminal reward without reaching the intended state.

Outcome supervision evaluates the final result, while process supervision additionally evaluates intermediate reasoning. Process supervision has shown important empirical advantages in mathematical reasoning (Lightman et al., 2023). More recently, agentic verifiers have expanded verification into a multi-turn process with tools, forward/backward checking, and active execution. AgentV-RL, for example, transforms reward modeling into tool-augmented deliberation and reports substantial gains over outcome reward models (Zhang et al., 2026). In competitive programming, agentic verification has also been used to actively search for discriminative test inputs that expose behavioral disagreement among candidate programs (Ma et al., 2026).

These developments motivate a stronger question:

What evidence is sufficient to justify reward when observational success can be produced by multiple incompatible mechanisms?

Our answer is that verification should be *interventional*. Instead of only inspecting a submitted trajectory, the verifier changes variables that the candidate’s success should depend on, re-runs the system, and asks whether behavior changes in the expected way. This shift is important because observational correctness and mechanistic reliability are not equivalent.

Consider two policies, π_f and π_s , evaluated on one task instance x . Suppose both return the correct answer:

$$\pi_f(x) = \pi_s(x) = y^*.$$

An outcome verifier receives identical evidence for the two candidates. Yet π_f may actually implement the task, while π_s may memorize y^* or ignore a necessary input. If a targeted intervention I changes a relevant variable, the policies may separate:

$$\pi_f(I(x)) = y_I^*, \quad \pi_s(I(x)) \neq y_I^*.$$

Verification has now collected evidence that could not be obtained from the original observation alone.

We call this paradigm **Interventional Agentic Verification (IAV)**. IAV has three core components:

1. **Hypothesis construction:** represent what the candidate claims to depend on and what shortcut alternatives remain plausible.
2. **Discriminative intervention selection:** choose perturbations expected to distinguish these hypotheses under a limited verification budget.
3. **Counterfactual execution and falsification:** apply interventions, compare predicted and observed consequences, and assign reward only after the candidate survives sufficiently informative tests.

The paper studies this idea in controlled executable environments. This is deliberate. Rather than entangling verifier quality with the generation quality, scale, prompting behavior, or proprietary access conditions of a particular language model, we isolate the verifier’s statistical and causal problem. We build IAV-BENCH, a multi-domain testbed in which shortcut mechanisms are known by construction and interventions are executable.

Our main contributions are:

- We formalize an **observational non-identifiability** problem for reward verification and give a sufficient interventional separability condition.
- We introduce **IAV**, which treats verification as active falsification under a finite intervention budget rather than passive classification.
- We introduce **discriminative witness selection**, showing empirically that arbitrary perturbations can fail even when interventions are available.
- We construct **IAV-Bench**, covering static reasoning corruption, shortcut policies, symbolic logic, executable code, and explicit reward-hacking environments.
- We evaluate IAV using multi-seed accuracy, false-positive rate, budget scaling, OOD transfer, Best-of- N , calibration, verifier noise, paired significance tests, and actual REINFORCE optimization.

2 Related Work

Outcome and process verification. Outcome reward models judge completed solutions, while process reward models score intermediate reasoning. Lightman et al. (2023) showed that process supervision can outperform outcome supervision in mathematical reasoning and released PRM800K. Our work is orthogonal: a process trace can itself be observationally plausible yet behaviorally disconnected from the mechanism producing the output. IAV asks whether claimed dependencies survive interventions.

Agentic verifiers. AgentV-RL uses multi-turn planning, forward/backward validation, tool use, and RL to improve reward modeling (Zhang et al., 2026). This is a major step beyond single-pass judging. IAV differs by making *intervention on candidate-relevant variables* the central verification primitive. Inspection asks whether a submitted trajectory is internally and externally consistent; intervention asks whether the candidate behaves correctly in nearby worlds designed to falsify it.

Counterexample generation and executable checking. Recent work in competitive coding actively searches for discriminative inputs that expose behavioral differences between candidate programs (Ma et al., 2026). Formal counterexample generation has also been studied for theorem proving, including automatically verifiable Lean counterexamples (Li et al., 2026). These works motivate active falsification. Our focus is broader: a domain-general verification abstraction, explicit observational-identifiability analysis, finite-budget witness selection, reward-hack repair interventions, and downstream reward optimization.

Reward hacking. Reward hacking and corrupted reward channels are longstanding alignment problems (Everitt et al., 2017). Hack-Verifiable Environments recently proposed embedding detectable hacking opportunities directly into environments so exploitation can be measured deterministically (Roth et al., 2026). We take a complementary perspective: given observed reward, can the verifier actively repair or perturb the environment to determine whether the success depended on an exploit?

3 Problem Formulation

Let $\mathcal{X}$ be a task space, $\mathcal{Y}$ an outcome space, and $\pi : \mathcal{X} \rightarrow \mathcal{Y}$ a policy, program, agent, or solver. For a task x , let $g(x)$ denote the intended correct outcome.

A conventional verifier observes

$$o = (x, \pi(x))$$

and returns a score

$$V_{\text{obs}}(o) \in [0, 1].$$

We distinguish *observational success*

$$S_{\text{obs}}(x, \pi) = \mathbb{I}[\pi(x) = g(x)]$$

from *behavioral reliability* under an intervention family $\mathcal{I}$:

$$S_{\text{rel}}(x, \pi; \mathcal{I}) = \mathbb{E}_{I \sim \mathcal{I}} [\mathbb{I}[\pi(I(x)) = g(I(x))]].$$

Definition 1 (Observational equivalence). *Two policies π_1, π_2 are observationally equivalent at x if*

$$\pi_1(x) = \pi_2(x).$$

Proposition 1 (Observational non-identifiability). *Let V_{obs} be any verifier whose input is restricted to $(x, \pi(x))$. If two policies π_1 and π_2 are observationally equivalent at x , then V_{obs} cannot distinguish them at x .*

Proof. Since $\pi_1(x) = \pi_2(x) = y$, the verifier receives exactly the same input tuple (x, y) for both policies. Therefore its output distribution is identical for π_1 and π_2 . Any distinction would require additional evidence not contained in the observation. $\square$

This elementary result becomes consequential under optimization: if a faithful policy and a shortcut policy are observationally equivalent on the training distribution, outcome reward does not provide a learning signal that prefers the faithful mechanism.

Definition 2 (Interventional separability). *Policies π_1, π_2 are separable under intervention family $\mathcal{I}$ at x if there exists $I \in \mathcal{I}$ such that*

$$\pi_1(I(x)) \neq \pi_2(I(x)).$$

Proposition 2 (Sufficient witness). *Assume the intended oracle $g(I(x))$ is executable and exact. If π_f is reliable under intervention I and π_s is not, i.e.*

$$\pi_f(I(x)) = g(I(x)), \quad \pi_s(I(x)) \neq g(I(x)),$$

then a verifier that executes I and compares against $g(I(x))$ perfectly distinguishes π_f and π_s on that witness.

The practical problem is therefore not merely whether interventions exist, but which interventions are worth spending verification compute on.

Proposition 3 (Finite-family coverage). *Let $\mathcal{H}_s$ be a finite family of shortcut hypotheses and let $\mathcal{S} \subseteq \mathcal{I}$ be a selected intervention set. If for every $\pi_s \in \mathcal{H}_s$ there exists $I \in \mathcal{S}$ such that*

$$\pi_s(I(x)) \neq g(I(x)),$$

then a deterministic verifier that rejects upon the first failed intervention has zero false-positive rate over $\mathcal{H}_s$.

This turns finite-budget verifier design into a coverage problem: each intervention “covers” the shortcuts it can falsify, while cost-aware verification seeks high shortcut coverage per unit execution cost.

Proposition 4 (Miss probability under noisy repeated testing). *Suppose each independently selected test exposes a fixed shortcut with probability at least q , and an exposing test is independently flipped to a false pass with probability at most ϵ . Then after B tests,*

$$P(\text{miss}) \leq (1 - q(1 - \epsilon))^B.$$

The bound is intentionally simple, but it motivates the budget and injected-noise studies below: reliability improves with discriminative-test probability and degrades smoothly with verifier noise.

The practical problem is therefore not merely whether interventions exist, but which interventions are worth spending verification compute on.

4 Interventional Agentic Verification

4.1 Verification as active falsification

Given candidate π , task x , verification history $h_{<t}$, and remaining budget $B - t$, IAV selects

$$I_t \sim q_\phi(I \mid x, \pi, h_{<t})$$

and observes

$$e_t = (I_t, \pi(I_t(x)), g(I_t(x))).$$

The verifier aggregates evidence

$$h_t = h_{t-1} \cup \{e_t\}$$

and returns reward

$$R_{\text{IAV}}(\pi, x, h_B) \in [0, 1].$$

Unlike a static process checker, IAV can create new evidence.

4.2 Discriminative intervention selection

A perturbation is useful only if it changes the posterior over competing hypotheses. Let Z denote latent candidate type (e.g., faithful, memorized, input-ignoring, exploit-based). We define intervention utility

$$U(I) = \frac{\mathbb{E}[H(Z \mid h) - H(Z \mid h, e_I)]}{C(I) + \epsilon},$$

where $C(I)$ measures verification cost.

The ideal next intervention is

$$I^* = \arg \max_{I \in \mathcal{I}} U(I).$$

Our executable experiments instantiate this principle with simple exact witnesses or structural heuristics. The negative-result experiments in Section 7.3 show why this choice matters.

4.3 Counterfactual causal credit

For an intermediate claim or dependency c_i , define an intervention $do(c_i \leftarrow \tilde{c}_i)$. We assign counterfactual contribution

$$C_i = \mathbb{E}_{\tilde{c}_i} [R(\tau) - R(\tau_{do(c_i \leftarrow \tilde{c}_i)})].$$

This should not be interpreted as recovering the model’s hidden neural causal graph. It is an operational statement about whether the *submitted solution structure* materially supports the rewarded outcome.

4.4 Falsification certificates

Instead of emitting only a scalar, IAV can return

$$\mathcal{F} = \{\text{candidate, interventions, predicted effects, observed effects, violations, confidence}\}.$$

Such certificates support auditing and make it possible to distinguish “correct answer” from “correctly supported answer.”

5 IAV-Bench: Executable Verification Benchmark

We construct five controlled suites. In each, the ground-truth mechanism and shortcut family are known, allowing exact verification of false positives.

5.1 Static reasoning corruption

Five-step arithmetic derivations are generated from executable dependency graphs. Corruptions include wrong final answers, wrong intermediate steps with correct final answers, missing dependencies, spurious intermediates, and two-step corruptions. This suite tests whether a verifier can reject reasoning that ends at the right answer for the wrong structural reason.

5.2 Counterfactual shortcut policies

All candidate policies are correct on the observed instance. Faithful policies implement the target function, while shortcut policies memorize the original answer or ignore one of five task variables. This suite directly realizes Proposition 1: outcome evidence is intentionally non-identifying.

5.3 Symbolic logic

Boolean rule chains are evaluated under three input premises. Shortcut policies memorize the observed terminal state or ignore one premise. Reliability is defined extensionally over the full truth table. Because some perturbations are non-discriminative, this domain provides a clean test of witness selection.

5.4 Executable code

Candidate programs implement

$$f(x, y, z) = (x + y)z - y.$$

Shortcut implementations memorize the observed output, ignore one input, or contain a guarded bug that only passes the original input. Verification executes perturbed inputs.

5.5 Reward-hack environments

We construct deterministic terminal-reward loopholes: `magic_submit`, `stale_state`, and `premature_done`. All exploit policies receive observed reward. Interventional verification “repairs” one environment mechanism at a time and replays the candidate, analogous in spirit to making reward hacking verifiable by design (Roth et al., 2026).

5.6 Adversarial unknown-shortcut suite

To remove the assumption that the verifier is told which failure family to test, we add a finite but heterogeneous adversarial suite with 42 candidate mechanisms. The suite contains faithful behavior, memorization, single- and two-variable dependency dropping, conditional shortcuts that activate only under a trigger intervention, and parity-dependent shortcuts that fail only on a restricted perturbation class. All candidates are observationally correct before verification.

The verifier receives only a hypothesis family and binary pass/fail evidence from executed interventions. It is *not* given the sampled shortcut identity. The intervention pool contains 57 single-, pair-, triple-, and joint-variable tests with nonuniform costs. This setting supports exact computation of intervention response partitions and therefore allows information-gain selection without an external model.

Table 1: Controlled verification results. Mean across eight seeds. Perfect scores occur in deliberately constructed executable suites and should not be interpreted as open-world performance.

Domain	Method	Budget	Accuracy $\uparrow$	95% CI	FPR $\downarrow$
Counterfactual policy	Outcome	0	0.344	[0.330, 0.357]	1.000
Counterfactual policy	Random interventions	3	0.799	[0.790, 0.809]	0.306
Counterfactual policy	IAV joint witness	1	1.000	[1.000, 1.000]	0.000
Executable code	Outcome	0	0.376	[0.364, 0.388]	1.000
Executable code	Targeted	2	0.879	[0.871, 0.887]	0.194
Executable code	IAV joint witness	1	1.000	[1.000, 1.000]	0.000
Symbolic logic	Outcome	0	0.403	[0.393, 0.412]	1.000
Symbolic logic	Adaptive witness	2	0.932	[0.926, 0.937]	0.115
Symbolic logic	Adaptive witness	3	1.000	[1.000, 1.000]	0.000
Static reasoning	Outcome	0	0.413	[0.403, 0.424]	0.835
Static reasoning	Random check	1	0.699	[0.686, 0.712]	0.428
Static reasoning	Adaptive check	1	1.000	[1.000, 1.000]	0.000

6 Experimental Setup

Seeds and reporting. Primary verification results are averaged across eight independent seeds with 95% normal-approximation confidence intervals over seed-level metrics. Best-of- N uses 20 seeds. REINFORCE uses 20 independent stochastic optimization runs.

Metrics. We emphasize accuracy and false-positive rate (FPR), since false-positive reward is directly exploitable. We also report F1, calibration (Brier score and expected calibration error), Best-of- N reliable-selection rate, unseen-perturbation success, and verifier-noise robustness.

Baselines. Controlled baselines include outcome-only verification, random process checking, random interventions, fixed structural/sensitivity heuristics, arbitrary joint perturbation, adaptive witness selection, exact information-gain selection, and cost-aware falsification-gain selection. For the unknown-shortcut suite, information gain is computed from the partition of the current version space induced by each executable intervention; the verifier never observes the true shortcut identity. Random schedules are averaged over 5,000 sampled intervention orders. The experiments intentionally isolate verification strategy from LLM generation quality.

7 Results

7.1 Main verification results

Table 1 summarizes the primary controlled results.

The intentionally confounded suites expose the weakness predicted by Proposition 1. Outcome verification accepts every observationally successful shortcut in the policy, code, and logic domains, producing FPR=1.0. Interventional evidence makes these classes distinguishable.

7.2 Verification budget scaling

Figure 1 shows that random interventions improve as budget increases, but remain less efficient than a discriminative joint witness. With three random interventions, counterfactual-policy accuracy reaches 79.9% and FPR falls to 30.6%; an informative joint intervention solves the controlled identification problem with one test.

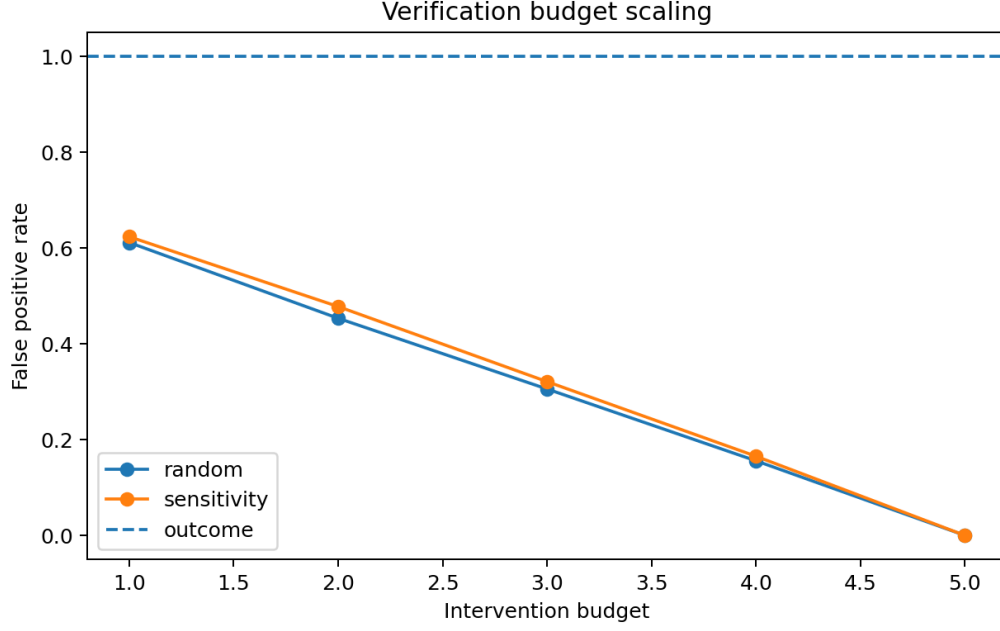


Figure 1: False-positive rate versus intervention budget in the counterfactual-policy suite.

7.3 Arbitrary intervention is not enough

A key negative result appears in symbolic logic. Our first “joint perturbation” simply flipped all premises. It achieved only 70.0% accuracy and retained FPR=50.2%. The reason is structural: many flips do not distinguish faithful and shortcut policies because the target function can be invariant on that perturbation.

We corrected the evaluation in two ways. First, reliability labels were defined by full functional equivalence over the truth table. Second, interventions were selected as *witness pairs*: input pairs differing in a variable for which the true function output also differs. Adaptive witnesses achieve 93.2% accuracy at budget two and 100% at budget three.

This result is central to the paper:

$$\text{intervention} \neq \text{discriminative intervention.}$$

The verifier must seek evidence that separates hypotheses, not merely perturb inputs.

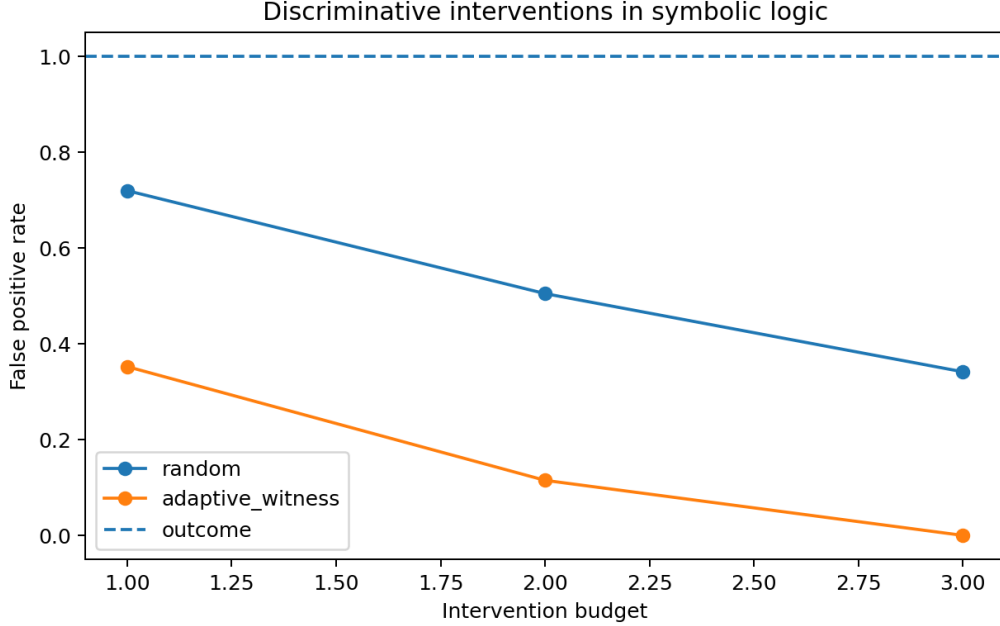


Figure 2: Random versus discriminative interventions in symbolic logic.

7.4 Unknown shortcut identity: adaptive falsification

A stronger test removes the assumption that the verifier knows which failure family is present. We evaluate a 42-hypothesis candidate family and a 57-intervention pool. Every shortcut has at least one executable witness, but hard conditional and parity shortcuts require the verifier to choose the right perturbation class.

Under one test, a random intervention obtains 54.3% expected accuracy with FPR 70.3%. Exact information-gain selection chooses a joint trigger intervention and reaches 87.0% accuracy with FPR 20.0%. With two adaptively chosen tests, information-gain selection covers the entire controlled shortcut family and reaches 100% accuracy, while two random tests reach 62.3% expected accuracy and FPR 58.0%. The candidate-specific oracle also achieves perfect classification with one test, providing an upper bound rather than a deployable baseline.

7.5 Cost-aware verification

Interventions have nonuniform execution costs. At total verification cost 3, the adaptive high-gain test reaches 87.0% accuracy and FPR 20.0%, compared with approximately 59.9% accuracy and FPR 61.7% for random schedules at comparable expected cost. At total cost 6, two high-gain interventions cover all controlled shortcut hypotheses, whereas random schedules retain FPR 46.8%.

A naive information-gain-per-cost heuristic is not uniformly superior to high absolute falsification gain: very cheap single-variable tests can be locally efficient yet fail to cover conditional shortcuts. We retain this negative result in the appendix and use it to distinguish *cheap* evidence from *globally discriminative* evidence.

7.6 Co-evolving shortcuts

Static exploit families can underestimate verifier pressure because optimization shifts probability mass toward whichever shortcuts survive current tests. We therefore run a controlled verifier–adversary co-evolution experiment. The adversary reweights shortcut mechanisms toward those surviving the persistent verifier test set and injects mutation mass each round. An adaptive verifier chooses the intervention with maximum expected falsification under the current adversary distribution; the random baseline chooses an intervention uniformly.

The adaptive verifier reduces exploit escape mass from 20.0% after its first test to 0% after its second persistent test in this finite hypothesis family. By contrast, the random verifier’s escape rate remains above 70% and increases as the adversary concentrates on blind spots. This is not a claim about open-ended neural red teaming; it is an executable demonstration that a fixed/random verifier can create selection pressure toward its own untested regions.

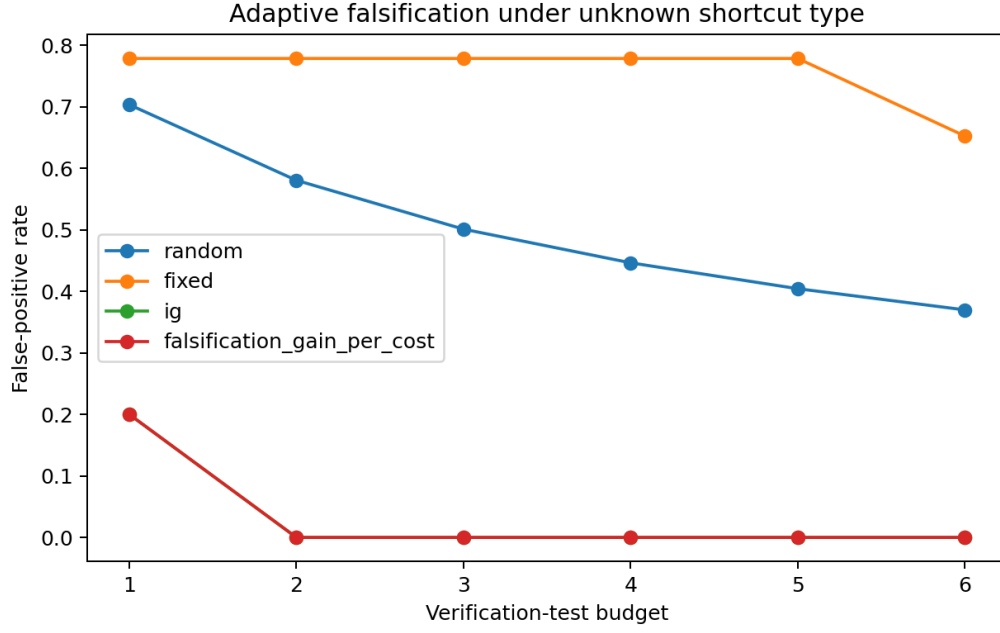


Figure 3: False-positive rate when the verifier is not told which shortcut family generated the observationally correct candidate. Information-gain selection uses only the current hypothesis version space and executable test responses.

7.7 Best-of- N selection

The verifier’s purpose is not only classification; it must improve decisions among candidates. Figure 6 reports selection performance. At $N = 8$, outcome scoring selects a reliable policy 35.1% of the time. Three random interventions increase this to 63.2%, while joint IAV reaches 96.8%. On a held-out perturbation not used by the selector, success rises from 75.2% under outcome scoring to 98.7% under joint IAV. At $N = 16$, reliable selection reaches 99.9%.

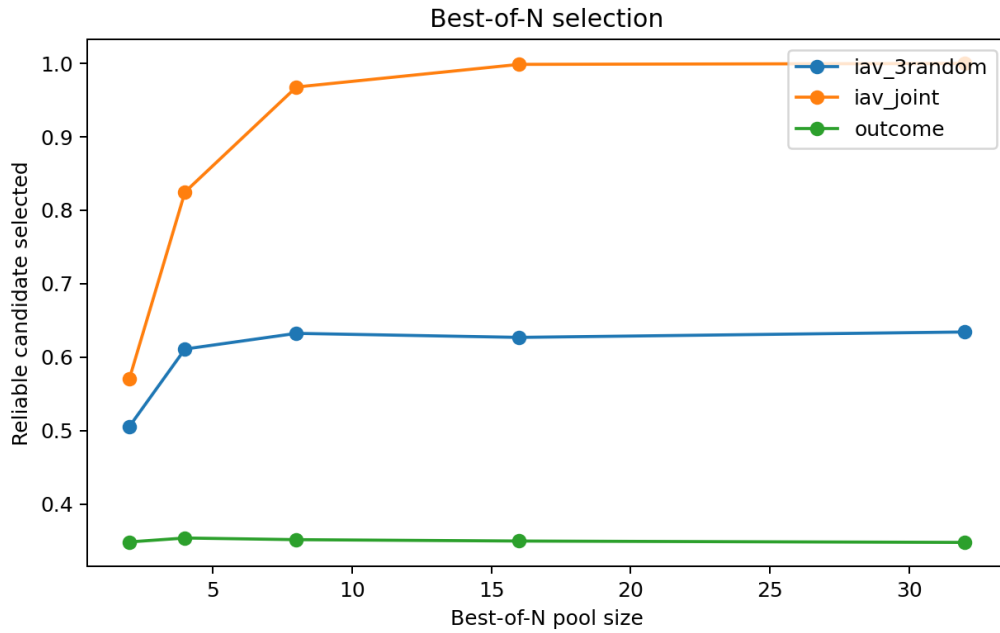


Figure 6: Reliable candidate selection as the candidate pool grows.

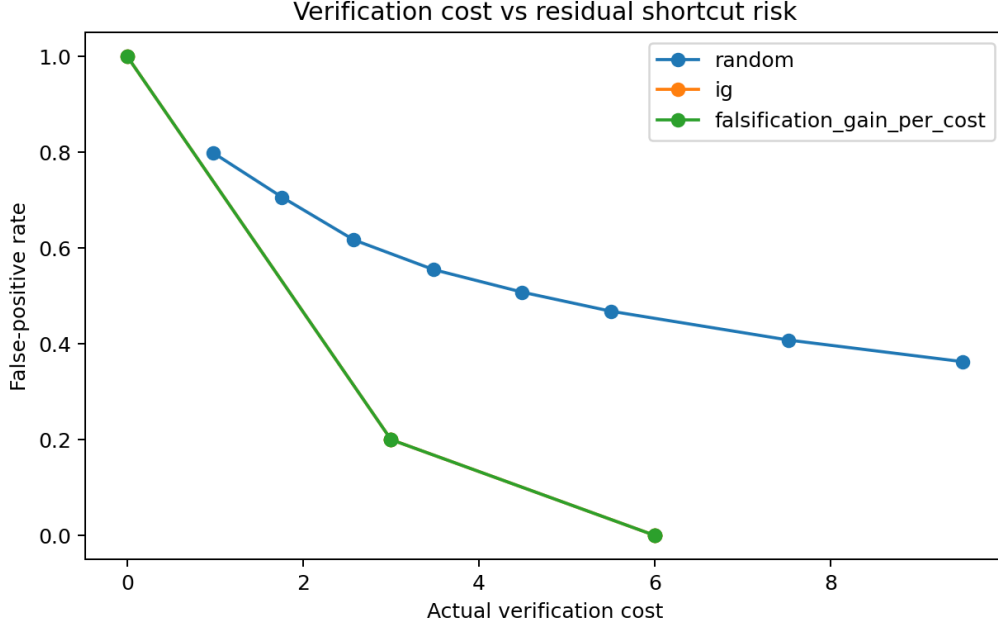


Figure 4: Residual shortcut false-positive rate as a function of actual verification cost.

7.8 Reward-hack detection

In the reward-hack suite, all exploit candidates receive the observed terminal reward. Outcome and shallow static-process verification therefore each have 35.8% accuracy and 100% FPR. One targeted repair intervention is sufficient to achieve perfect classification in this deterministic controlled setting. Random repairs scale predictably: accuracy rises from 57.5% at budget one to 78.6% at budget two and 100% at budget three.

The important claim is not the perfect score itself, but the mechanism: the verifier tests whether reward survives after the loophole responsible for apparent success is removed.

7.9 Downstream policy-gradient optimization

We next test whether verifier quality changes the optimization incentive. We parameterize a categorical stochastic policy over seven solver strategies: one faithful policy and six shortcut policies. Initial faithful probability is 35%. We optimize the policy logits with actual REINFORCE updates using either outcome reward, three random IAV tests, or a joint IAV witness.

After 800 updates, faithful probability is:

$$0.366 \quad \text{Outcome}, \quad 0.987 \quad \text{Random-3 IAV}, \quad 0.996 \quad \text{Joint IAV}.$$

Across 20 runs, the 95% CI is [0.312, 0.420] for outcome reward and [0.9959, 0.9971] for joint IAV.

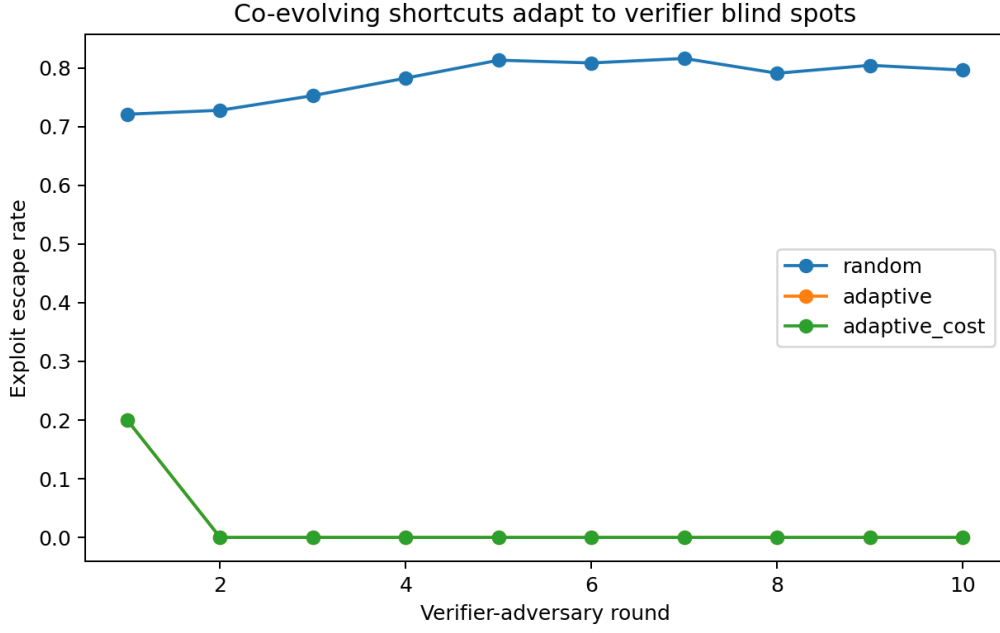


Figure 5: Controlled co-evolution. Shortcut mass shifts toward verifier blind spots; adaptive persistent tests close those blind spots substantially faster than random testing.

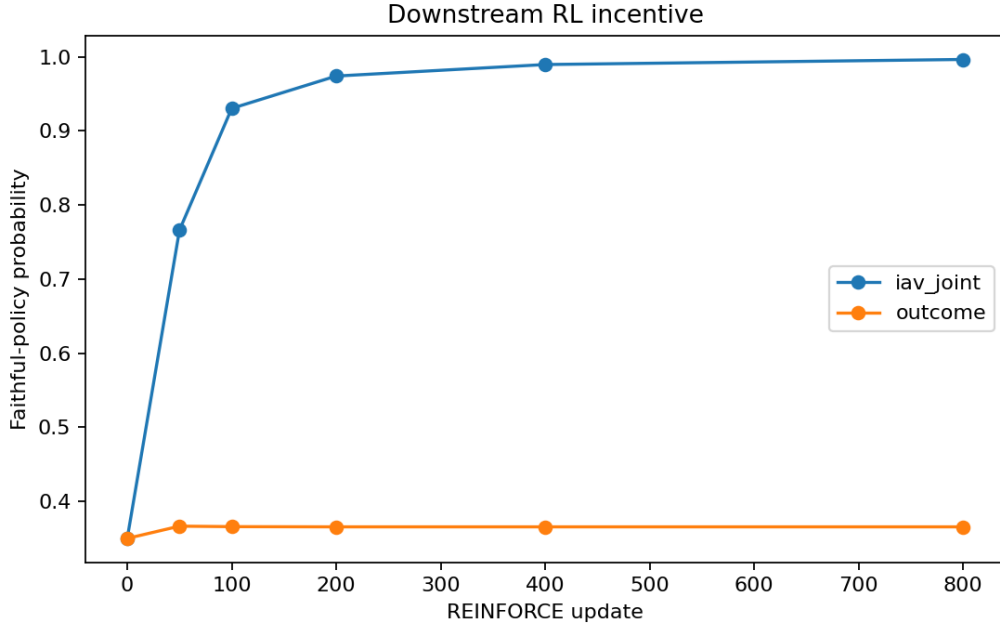


Figure 7: Actual stochastic policy-gradient optimization over controlled solver strategies.

Outcome reward cannot reliably prefer the faithful strategy because shortcuts receive the same observational reward. Interventional reward supplies the missing discrimination signal.

7.10 Robustness, OOD, and calibration

OOD. We widen arithmetic ranges for the policy and code suites and lengthen symbolic logic chains from 8 to 14. The controlled joint witnesses retain perfect accuracy in the widened policy and code suites; adaptive symbolic

witnesses retain perfect classification at sufficient budget. These are controlled distribution shifts, not evidence of open-world generalization.

Intervention magnitude. Joint interventions remain perfect for deltas $-1, 1, 2, 5, 10$ and achieve about 97.1% accuracy for deltas -3 and -5 . This exposes another practical issue: interventions should avoid accidental collisions in which a shortcut’s stale value happens to coincide with the perturbed target behavior.

Verifier noise. We inject symmetric errors into intervention outcomes. Accuracy degrades smoothly: 94.7% at 5% noise, 90.4% at 10%, and 79.9% at 20%.

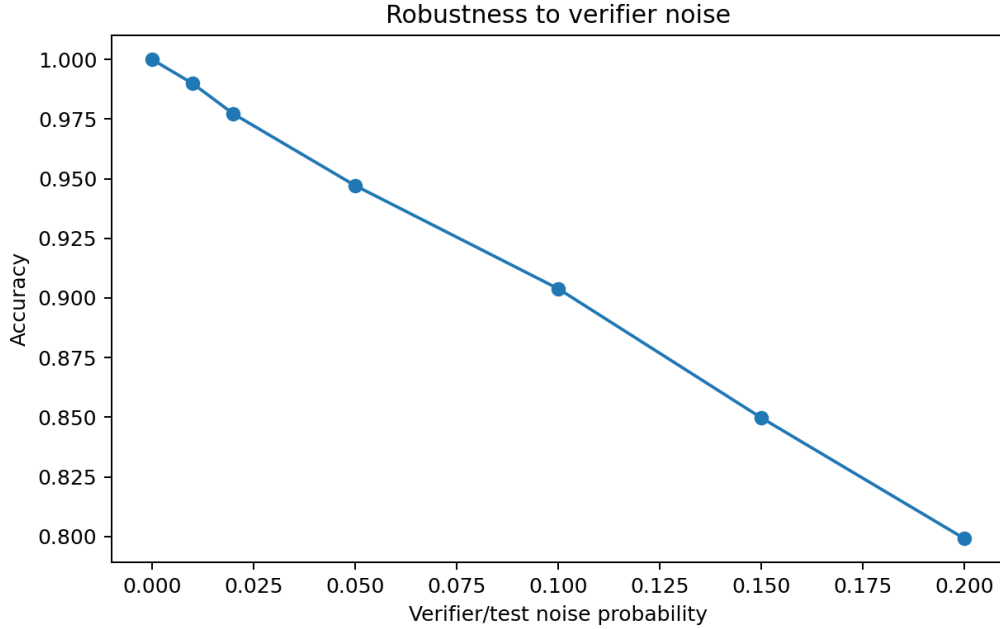


Figure 8: Robustness to symmetric verifier/test noise.

Calibration. On the counterfactual-policy suite, outcome scoring has Brier score 0.650 and ECE 0.650. Random-3 IAV improves these to 0.332 and 0.399. Joint IAV is perfectly calibrated on the deterministic controlled suite because the witness exactly separates the policy classes.

Paired significance. Across eight paired datasets, joint IAV corrects roughly 915–1016 examples per seed that outcome verification gets wrong and introduces zero paired regressions. Exact paired binomial p -values range from approximately 10^{-276} to 10^{-306} . These values should be interpreted in light of the intentionally confounded benchmark design: the experiment tests an identification property, not a subtle natural-data effect.

8 Discussion

8.1 What the results establish

The experiments support four claims.

- 1. Observational reward can be non-identifying.** When faithful and shortcut policies agree on the observed task, an outcome verifier has no evidence with which to distinguish them.
- 2. Interventions create new verification evidence.** By changing candidate-relevant variables and re-executing behavior, the verifier can expose differences hidden at the original observation.

3. Intervention quality matters. The symbolic-logic failure of arbitrary joint perturbation is especially important. The correct objective is not “perturb more,” but “choose tests with high expected discriminatory value.”

4. Verification changes optimization incentives. The REINFORCE experiment demonstrates a downstream consequence: when shortcuts and faithful behavior share the same observational reward, policy optimization can remain indifferent. Interventional reward changes the landscape so that faithful behavior is preferentially reinforced.

5. The verifier must adapt to the shortcut distribution. The unknown-shortcut and co-evolution experiments show that a verifier that spends tests without considering hypothesis disagreement can induce its own blind spots. Information-seeking selection converts the problem from “apply perturbations” to “allocate verification compute where surviving hypotheses disagree.”

6. Cost and information are distinct objectives. Cheap tests are not automatically useful tests. The cost-aware ablation shows that locally attractive low-cost perturbations can miss conditional failures that a more expensive joint witness exposes immediately. Practical verifier agents therefore need an explicit utility that trades off expected falsification value against tool cost rather than minimizing cost in isolation.

8.2 Relationship to LLM verification

IAV is compatible with language-model agents, but this paper deliberately avoids making frontier-model performance the central empirical claim. A language model can instantiate the candidate policy, the hypothesis generator, the intervention proposer, or the final evidence aggregator. However, those choices introduce additional variables—model scale, prompting, sampling, tokenizer effects, tool reliability, and training infrastructure—that are not required to test the verifier-identifiability hypothesis.

The controlled benchmark therefore asks a more basic question: *when observational evidence is insufficient, does active intervention resolve the ambiguity?* The answer is yes in the executable settings studied here, subject to the quality of the intervention family.

8.3 From intervention to verifier agents

A practical agentic implementation can use an LLM to:

1. decompose a solution into claims or environment dependencies;
2. maintain hypotheses about possible shortcuts or exploits;
3. propose candidate interventions;
4. estimate information gain per tool call;
5. execute tools or simulators;
6. aggregate evidence into a falsification certificate.

This architecture complements agentic reward modeling (Zhang et al., 2026): rather than replacing forward/backward inspection, interventional testing adds a new source of evidence.

9 Limitations

The paper has deliberate scope boundaries.

First, the current experiments use controlled executable policies, programs, and synthetic environments rather than sampled chain-of-thought from large pretrained language models. We therefore do *not* claim improvements on MATH-500, GSM8K, HumanEval, or neural RL with Qwen/Llama.

Second, several suites contain exact executable oracles. Real-world tasks may have partial, noisy, expensive, or unavailable ground truth. Our noise experiments partially address this issue but do not solve it.

Third, the strongest controlled results rely on intervention families that contain discriminative witnesses. If the verifier cannot manipulate a causally relevant variable, or if all accessible interventions leave candidate hypotheses observationally equivalent, the identification problem remains.

Fourth, the strongest adaptive-selector experiment uses a finite executable hypothesis family, so information gain can be computed exactly. This is a feature for isolating the verification problem, but open-ended environments require approximate hypothesis generation and value-of-information estimation. The reward-hack transfer experiment in the appendix also shows a useful boundary: in symmetric domains, a generic selector may tie a cost-matched random policy at low budgets because the available tests are exchangeable.

Fifth, adaptive intervention selection is instantiated here using exact or lightweight structural heuristics. Learning a general intervention policy for open-ended environments is an important next step.

Sixth, counterfactual causal credit refers to the submitted task structure and observable policy behavior. It should not be interpreted as identifying the hidden neural causal mechanism of a language model.

These limitations are also an advantage of the present design: the paper makes a narrow, falsifiable claim about verification itself and evaluates that claim without relying on inaccessible model weights or proprietary inference APIs.

10 Conclusion

Reward should not be assigned merely because a candidate succeeds once. When multiple mechanisms produce the same observation, observational verification is fundamentally unable to identify which mechanism generated the success. Interventional Agentic Verification addresses this problem by creating new evidence: change variables that correct behavior should depend on, re-execute the candidate, and attempt to falsify it before rewarding it.

Across controlled reasoning, logic, code, shortcut-policy, and reward-hack environments, interventional verification sharply reduces false-positive reward, improves Best-of- N selection, and changes policy-gradient incentives toward faithful behavior. The most important negative result is equally clear: perturbation alone is not enough. Reliable verification requires *discriminative* interventions that separate plausible hypotheses under a finite budget.

The broader principle is simple:

Do not only ask whether the agent succeeded. Ask whether the success survives a test designed to break it.

References

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step. *arXiv:2305.20050*, 2023.
- Jiazheng Zhang, Ziche Fu, Zhiheng Xi, Wenqing Jing, Mingxu Chai, Wei He, Guoqiang Zhang, Chenghao Fan, Chenxin An, Wenxiang Chen, Zhicheng Liu, Haojie Pan, Dingwei Zhu, Tao Gui, Qi Zhang, and Xuanjing Huang. AgentV-RL: Scaling Reward Modeling with Agentic Verifier. *Findings of ACL 2026*, pages 23078–23100, 2026.
- Amit Roth, Ankur Samanta, Matan Halevy, Yoav Levine, and Yonathan Efroni. Hack-Verifiable Environments: Towards Evaluating Reward Hacking at Scale. *arXiv:2605.20744*, 2026.
- Zeyao Ma, Jing Zhang, Xiaokang Zhang, Jiaxi Yang, Zongmeng Zhang, Jiajun Zhang, Yuheng Jing, Lei Zhang, et al. Scaling Agentic Verifier for Competitive Coding. *arXiv:2602.04254*, 2026.
- Zenan Li, Zhaoyu Li, Kaiyu Yang, Xiaoxing Ma, and Zhendong Su. Learning to Disprove: Formal Counterexample Generation with Large Language Models. *arXiv:2603.19514*, 2026.
- Tom Everitt, Victoria Krakovna, Laurent Orseau, Marcus Hutter, and Shane Legg. Reinforcement Learning with a Corrupted Reward Channel. *arXiv:1705.08417*, 2017.
- DeepMind. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020.

A Benchmark Generation Details

Static arithmetic. Inputs are sampled independently from bounded integer ranges. The executable dependency chain is

$$s_0 = a + b, s_1 = s_0 c, s_2 = s_1 - d, s_3 = s_2 + e, s_4 = 2s_3.$$

Corruptions modify one or two non-terminal steps, delete dependencies, or alter only the final answer.

Counterfactual policies. The target is the same arithmetic function. Shortcut policies either return the original answer or replace one perturbed variable with its original value before recomputation.

Symbolic logic. Three Boolean premises drive an alternating AND/XOR rule chain. Reliability is evaluated over all 2^3 inputs. Witnesses are chosen by finding premise flips that change the true function output.

Code. The executable target is $f(x, y, z) = (x + y)z - y$. Candidate families include memorization, input ignoring, and a guarded off-by-one bug.

Reward hacks. Each exploit family is paired with a repair intervention that removes the corresponding loophole. Targeted verification chooses the repair suggested by the observed failure signature; random verification samples repairs.

B Additional Best-of- N Results

At $N = 2, 4, 8, 16, 32$, joint IAV reliable-selection rates are 0.570, 0.825, 0.968, 0.999, and 1.000 respectively. Outcome selection remains near the base faithful-policy prevalence (approximately 0.35) because all candidates are observationally correct.

C Adversarial Unknown-Shortcut Benchmark Details

The adversarial suite contains 42 hypotheses: one faithful policy, one memorization policy, five single-variable dependency-dropping policies, ten two-variable dependency-dropping policies, twenty conditional shortcuts, and five parity-dependent shortcuts. The intervention pool contains 57 tests: signed single-variable perturbations, pair perturbations, sparse triples, and two full joint interventions.

A conditional shortcut activates only when a designated trigger variable is moved by at least two units and a separate ignored variable is also changed. A parity shortcut fails only when the total intervention magnitude changes parity while touching its designated ignored variable. These mechanisms ensure that many intuitive one-variable tests are non-discriminative.

C.1 Exact information-gain selector

Let $\mathcal{V}_t$ denote the current version space after t passing tests. For candidate intervention I , each hypothesis predicts binary outcome $o_I(h) \in \{0, 1\}$. Under a normalized prior over $\mathcal{V}_t$, the selector computes

$$\text{IG}(I) = H(P(o_I = 1)),$$

which equals expected posterior entropy reduction for a deterministic binary partition. The next test maximizes this quantity. After an observed pass, the version space is restricted to hypotheses predicting pass; after a failure, the candidate is rejected.

Under the default prior, the exact IG schedule selects `joint.2` followed by `joint.1`. The first test reduces shortcut FPR to 20%; the second covers the remaining conditional/parity family.

C.2 Cost-aware ablation

Intervention costs are 1.0 for single-variable tests, 1.6 for pairs, 2.1 for sparse triples, and 3.0 for joint tests. We evaluated random schedules, fixed cheap-first schedules, entropy-per-cost, raw falsification gain, and falsification-gain-per-cost.

The cheap-first strategy illustrates a failure mode: after five unit-cost tests it still has FPR 77.8% because its negative-direction single-variable perturbations do not activate conditional shortcuts. Greedy entropy-per-cost improves coverage gradually but requires cost 8 to reach zero FPR. In contrast, the two high-gain joint witnesses cost 6 total and cover the finite shortcut family exactly.

D Prior-Shift Transfer

The information-gain schedule is computed once under a default candidate prior and frozen. We then change the shortcut mixture to conditional-heavy, parity-heavy, ignored-dependency-heavy, and balanced distributions without retuning the selector. The same two-test schedule retains zero FPR under all five tested priors because it covers the complete finite shortcut family. By contrast, the fixed cheap-first schedule has FPR ranging from 64.0% to 89.5% at budget two.

This experiment tests distribution shift over shortcut prevalence, not new unseen mechanism classes.

E Generic Selector Transfer to Reward-Hack Repairs

We also apply the same version-space information-gain rule, without domain-specific learned parameters, to a separate finite reward-hack hypothesis family consisting of magic-submit, stale-state, premature-termination, and composite exploits. The generic selector chooses `repair_magic`, `fresh_state`, and `strict_done`. With one, two, and three unit-cost repairs, accuracy is 68%, 88%, and 100%, respectively.

A cost-matched random baseline is identical at these budgets because the three unit-cost single repairs are symmetric under the chosen prior. This negative transfer result is informative: generic information gain is not expected to dominate random testing when available interventions induce equivalent partitions. The method’s advantage arises when candidate tests differ in discriminative structure.

F Co-Evolution Protocol

The adversary maintains a distribution over shortcut hypotheses. After each verifier round, shortcut hypotheses that survive all persistent tests receive multiplicative fitness advantage, after which 8% uniform mutation mass is injected. The adaptive verifier chooses the test maximizing expected falsification of the current adversary distribution; the random baseline samples uniformly.

This protocol deliberately excludes neural function approximation so the selection dynamics are exactly auditable. It models the incentive effect of verifier blind spots: once a test is known, probability mass migrates toward mechanisms not covered by that test set.

G Full Experimental Matrix

The accompanying artifact contains:

- eight-seed primary verifier evaluations;
- twenty-seed Best-of- N experiments;
- twenty-seed REINFORCE runs;
- exact adaptive-selector budget curves;
- 5,000 random intervention schedules for adaptive-selector baselines;
- cost-budget curves;
- shortcut-family detection breakdowns;
- five prior-shift mixtures;
- reward-hack selector-transfer experiments;
- verifier–adversary co-evolution;
- perturbation-magnitude robustness;
- OOD range/chain-length shifts;
- calibration, Brier score, and ECE;

- paired exact significance tests;
- injected verifier-noise stress tests.

No result requiring external model inference, proprietary API access, or unavailable weights is needed for a core claim in this version.

H Theoretical Notes on Budgeted Verification

For a finite shortcut family, define the coverage of intervention I as

$$C(I) = \{h \in \mathcal{H}_s : o_I(h) = 0\}.$$

Selecting a set of interventions under budget B to maximize falsified shortcut prior mass is a budgeted coverage problem:

$$\max_{S \subseteq \mathcal{I}} \sum_{h \in \cup_{I \in S} C(I)} p(h) \quad \text{s.t.} \quad \sum_{I \in S} c(I) \leq B.$$

This formulation separates three quantities that are often conflated: test count, test execution cost, and expected falsification value.

For open-ended agentic verifiers, $p(h)$ and $C(I)$ are not known exactly. A language-model verifier can be viewed as approximating both: it proposes plausible failure hypotheses, predicts which interventions distinguish them, executes tests, and updates the posterior from observed evidence.

I Reproducibility Statement

All numerical results reported in this manuscript were generated from executable experiments in the associated artifact. The artifact contains aggregate and raw seed-level CSV files, figures, and a full results report. No Qwen/Llama, external inference API, MATH-500, GSM8K, HumanEval, LoRA, or GRPO result is represented as executed in this version.